

Classes, Objects, Constructors, and Encapsulation in C#

1. Introduction

C# is a modern, object-oriented programming language developed by Microsoft as part of the .NET ecosystem. Object-Oriented Programming (OOP) is central to C#, enabling developers to model real-world systems through reusable and maintainable code. This research focuses on four foundational OOP concepts in C#: **classes, objects, constructors, and encapsulation**.

2. Classes and Objects

2.1 Classes

A **class** is a blueprint or template used to define properties (data) and methods (behavior). It does not occupy memory until an object is created from it.

Example:

```
public class Car
{
    public string Brand;
    public int Speed;

    public void Drive()
    {
        Console.WriteLine("The car is driving.");
    }
}
```

2.2 Objects

An **object** is an instance of a class. It represents a real-world entity and occupies memory.

Example:

```
Car myCar = new Car();
myCar.Brand = "Toyota";
myCar.Speed = 120;
myCar.Drive();
```

2.3 Key Characteristics

- Classes define structure
- Objects represent actual data
- Multiple objects can be created from one class

3. Properties

3.1 Definition

Properties provide a controlled way to access class fields. They combine aspects of fields and methods.

3.2 Types of Properties

- **Auto-implemented properties**
- **Read-only / Write-only properties**

Example:

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; private set; }
}
```

3.3 Benefits

- Data validation
- Controlled access
- Cleaner syntax

4. Constructors

4.1 Definition

A **constructor** is a special method used to initialize objects when they are created. It has the same name as the class and no return type.

4.2 Types of Constructors

1. **Default Constructor**
2. **Parameterized Constructor**
3. **Copy Constructor (custom implementation)**

Example:

```
public class Car
{
    public string Brand;
    public int Speed;

    // Constructor
    public Car(string brand, int speed)
    {
        Brand = brand;
        Speed = speed;
    }
}
```

Usage:

```
Car car1 = new Car("Honda", 150);
```

4.3 Importance

- Ensures object initialization
- Reduces errors
- Supports overloading

5. Encapsulation

5.1 Definition

Encapsulation is the practice of hiding internal data and exposing only what is necessary. It is achieved using access modifiers and properties.

6. Access Modifiers

Access modifiers control the visibility of class members.

Types in C#:

Modifier	Description
public	Accessible from anywhere
private	Accessible only within the class
protected	Accessible within the class and derived classes
internal	Accessible within the same assembly
protected internal	Combination of protected and internal

7. Getters and Setters

7.1 Definition

Getters and setters are methods used to read and modify private fields.

Example:

```
public class BankAccount
{
    private double balance;

    public double Balance
    {
        get {return balance; }
        set
        {
            if (value >= 0)
                balance = value;
        }
    }
}
```

```
    }  
  }  
}
```

7.2 Advantages

- Data validation
- Data protection
- Improved maintainability

8. Encapsulation in Practice

Encapsulation combines:

- **Private fields**
- **Public properties**
- **Controlled access**

Example:

```
public class Student  
{  
    private int marks;  
  
    public int Marks  
    {  
        get { return marks; }  
        set  
        {  
            if (value >= 0 && value <= 100)  
                marks = value;  
        }  
    }  
}
```

9. Benefits of Encapsulation

- Improves security
- Reduces system complexity
- Enhances code maintainability
- Supports data hiding

10. Conclusion

Classes, objects, constructors, and encapsulation form the backbone of C# object-oriented programming. Classes define structure; objects represent instances, constructors initialize data, and encapsulation ensures secure and controlled access. Together, these concepts enable developers to build scalable, maintainable, and robust software systems.